Métricas de software en los diferentes enfoques de gestión

1. Punto de partida: ¿para qué medimos software?

Una métrica de software es una medida utilizada para observar alguna característica del producto, del proyecto o del proceso de desarrollo. Las métricas no son el objetivo del trabajo: son instrumentos que permiten entender qué está ocurriendo y decidir cómo mejorar.

La presentación distingue tres grandes enfoques de gestión: tradicional, basado en procesos definidos; ágil, asociado a procesos empíricos; y Lean/Kanban, que también trabaja de forma empírica pero pone especial atención al flujo de trabajo y a los tiempos de entrega.

La pregunta central es siempre qué queremos saber y qué decisión podríamos tomar con el dato. No existe una única métrica que sirva para todas las organizaciones, equipos o productos.

2. Métricas en el enfoque tradicional: procesos definidos

2.1. Tres dominios de métricas

- Proceso: reflejan el desempeño del proceso de desarrollo a lo largo del tiempo. Pueden utilizarse para mejorar prácticas que se repiten en varios proyectos.
- Proyecto: describen planificación y ejecución de un proyecto concreto: estimaciones, avance, fechas, esfuerzo, presupuesto y defectos.
- Producto: caracterizan al software producido: tamaño, cobertura y defectos, entre otras propiedades.

Según la presentación, las métricas de distintos proyectos pueden consolidarse para formar métricas de proceso que sean conocidas por la organización. Por ejemplo, varios proyectos pueden aportar datos sobre precisión de estimaciones o defectos para comprender tendencias organizacionales.

2.2. Las cuatro métricas básicas de un proyecto

- Tamaño del producto: dimensión del software que se construye. Puede expresarse, según el contexto, mediante líneas de código, requisitos, funcionalidades u otras unidades.
- Esfuerzo: trabajo dedicado por una o varias personas. No es lo mismo que tiempo calendario. Ejemplo: dos personas que trabajan 10 horas cada una representan 20 horas-persona de esfuerzo.
- Tiempo (calendario): duración transcurrida desde un hito hasta otro. Incluye el efecto de esperas, dependencias y secuencia de actividades.
- Defectos: errores o fallas identificados durante revisiones, pruebas e incluso después de una entrega. Importan tanto su cantidad como su tipo o gravedad.

> Para entenderlo: Una tarea puede requerir 16 horas-persona de esfuerzo y extenderse durante cuatro días calendario si solo se destinan cuatro horas diarias. Esfuerzo y duración miden cosas diferentes.

2.3. ¿Qué le interesa medir a cada participante?

La diapositiva «El sueño del pibe...» presenta una colección muy amplia de posibles mediciones. No implica que deban adoptarse todas: muestra que cada rol tiene preguntas diferentes.

Perspectiva	Mediciones que destaca la presentación
Desarrollador	Esfuerzo; esfuerzo y duración estimada versus real de cada tarea; porcentaje de cobertura de pruebas unitarias; cantidad y tipo de defectos en pruebas unitarias y en revisiones por pares.
Equipo de desarrollo	Tamaño del producto; duración de hitos; niveles de personal previstos y reales; tareas planificadas y terminadas; distribución del esfuerzo; estado y volatilidad de requisitos; defectos en integración, sistema y revisiones por pares; distribución de defectos; porcentaje de pruebas ejecutadas.
Organización	Tiempo calendario; desempeño de esfuerzo y presupuesto respecto de lo planificado; precisión de estimaciones de plazo y esfuerzo; defectos presentes en la versión liberada.

Estado de los requisitos indica qué se ha definido, implementado o validado. Volatilidad de los requisitos refiere a cuánto cambian mientras avanza el proyecto. Staffing es la dotación de personas asignadas; peer review es revisión por pares; y release significa versión entregada o liberada.

2.4. Mantener las métricas simples

La presentación propone una idea muy concreta: «Si estás a millas de distancia de tu destino, no tiene sentido medir en milímetros». La precisión y el detalle deben ser proporcionales a la necesidad real.

Antes de incorporar una métrica conviene responder tres preguntas: ¿aporta información adicional?, ¿tiene beneficio práctico?, ¿responde lo que queremos saber? Si no aporta a ninguna decisión, medirla agrega trabajo sin generar valor.

2.5. Balance entre necesidades, expectativas y restricciones

El esquema de la diapositiva 7 vincula las necesidades y expectativas de las partes interesadas con un equilibrio entre alcance, calidad, tiempo y recursos. Además muestra restricciones vinculadas con personas, tecnología y proceso.

La idea es que una decisión sobre una dimensión afecta a otras: ampliar el alcance puede presionar los tiempos o los recursos; acortar un plazo exige revisar prioridades, capacidad o riesgos. Las métricas ayudan a hacer visibles estas tensiones, pero por sí solas no resuelven el equilibrio.

> Para entenderlo: Si un cliente solicita cinco funcionalidades nuevas sin modificar fecha ni recursos, medir la variación de alcance y el esfuerzo estimado ayuda a conversar sobre las consecuencias de esa solicitud.

3. Métricas en ambientes ágiles: procesos empíricos

3.1. Regla de oro y principios

La regla de oro de la presentación es que «la medición es una salida, no una actividad»: se intenta obtener información útil a partir del trabajo real, sin transformar la medición en una carga independiente. La filosofía es minimalista: medir lo necesario y nada más.

Dos principios ágiles orientan la elección de indicadores: (1) priorizar la satisfacción del cliente mediante entregas tempranas y continuas de software valioso y funcionando; (2) considerar que el software funcionando es la principal medida de progreso.

Por eso, contar tareas iniciadas o tiempo ocupado no alcanza para afirmar que existe progreso. El equipo necesita observar cuánto trabajo efectivamente se termina y qué valor probado puede utilizarse.

3.2. Velocidad (Velocity)

La velocidad representa la cantidad de trabajo estimado que el equipo completa en una iteración o Sprint. En un contexto con puntos de historia, se expresa normalmente en puntos completados por Sprint; no son puntos iniciados ni parcialmente terminados.

Cálculo orientativo: Velocidad del Sprint = suma de puntos de historia de los elementos terminados según la definición de terminado del equipo.

En el gráfico de la diapositiva 11, la velocidad cambia de una iteración a otra; se observan valores como 25, 45, 35, 30, 42, 4, 43, 38, 41, 33, 44 y 40 puntos en las iteraciones con valores representados. El dato bajo de 4 puntos muestra por qué conviene interpretar una secuencia y no sacar conclusiones con una sola observación.

Ejemplo: si el equipo termina historias de 3, 5, 8 y 5 puntos, su velocidad es 21 puntos. Una historia de 8 puntos comenzada pero no terminada no suma todavía.

Uso: orientar previsiones del propio equipo con datos históricos. Precaución: los puntos de historia dependen de la escala de estimación del equipo; una velocidad de 40 no vuelve automáticamente superior a un equipo frente a otro de 25.

3.3. Capacidad

La capacidad expresa la disponibilidad de trabajo del equipo durante un Sprint, que suele estimarse en horas-persona disponibles. La tabla de la diapositiva 12 combina horas con puntos de historia para siete Sprints:

Sprint	1	2	3	4	5	6	7	Total
Horas	240	240	480	480	420	420	480	2760
Puntos de historia	30	30	45	60	58	52	60	335

El gráfico de capacidad exhibe solamente las horas, no los puntos de historia. Se observa que la disponibilidad pasa de 240 horas en los primeros Sprints a 480 en otros, y baja a 420 en los Sprints 5 y 6. Esto permite razonar por qué el compromiso de trabajo podría variar entre iteraciones.

La capacidad suele depender del número de integrantes, jornadas, vacaciones, ausencias y otras tareas. No equivale a velocidad: la primera alude a recursos disponibles; la segunda al volumen de trabajo estimado que se completó.

> Para entenderlo: Si cuatro personas cuentan con 30 horas efectivas de desarrollo cada una durante el Sprint, la capacidad planificada será 120 horas-persona. No se convierte automáticamente a puntos de historia sin conocer el comportamiento histórico del equipo.

3.4. Running Tested Features (RTF)

Running Tested Features significa, aproximadamente, funcionalidades en funcionamiento y probadas. El indicador busca seguir aquellas funcionalidades que ya están operativas y que pasaron las verificaciones exigidas para considerarlas útiles.

El gráfico de la diapositiva 13 representa su evolución por iteración: desde 0 asciende a 2, 3, 3, 4, 5, 6; luego desciende a 5 y vuelve a subir a 8, 9 y 10. La caída ilustra que la cantidad de funciones consideradas listas y funcionando puede disminuir si alguna deja de cumplir las condiciones necesarias.

Diferencia con velocidad: la velocidad usa unidades de estimación completadas durante una iteración; RTF pone el foco en funcionalidades realmente operativas y verificadas a lo largo del tiempo. Son miradas complementarias del progreso.

3.5. Cómo se relacionan las tres métricas

- Capacidad: ¿cuánto tiempo y disponibilidad tenemos para trabajar?
- Velocidad: ¿cuánto trabajo estimado terminamos efectivamente en cada iteración?
- RTF: ¿cuántas funcionalidades funcionan y fueron probadas?

No hay que usarlas como sinónimos. Una capacidad elevada no garantiza una velocidad elevada, y una velocidad elevada no garantiza que aumente proporcionalmente la cantidad de funcionalidades operativas.

4. Métricas Lean / Kanban: observar el flujo

La sección Kanban de la presentación cambia el centro de atención: además de considerar el trabajo realizado, interesa cuánto tarda cada elemento en circular desde su solicitud hasta la entrega y cuánto de ese tiempo se trabaja realmente.

4.1. Lead Time: mirada del cliente

El Lead Time es el tiempo transcurrido desde que se solicita un elemento de trabajo hasta que se entrega. La diapositiva lo identifica como «vista del cliente», porque refleja la espera completa que percibe quien hace el pedido.

Fórmula: Lead Time = fecha/hora de entrega − fecha/hora de solicitud. Es habitual expresarlo en días; debe mantenerse consistente el criterio de días calendario o hábiles utilizado.

4.2. Cycle Time: mirada interna

El Cycle Time mide el tiempo entre el inicio del proceso de trabajo para un ítem y su finalización. Se relaciona con la capacidad interna del proceso y con su ritmo de terminación.

Fórmula: Cycle Time = momento de finalización − momento de inicio del trabajo. Por lo general, la solicitud precede al inicio del trabajo, de modo que el Lead Time incluye un tramo de espera inicial que no forma parte del Cycle Time.

4.3. Touch Time: tiempo de trabajo efectivo

El Touch Time es el tiempo en que un ítem fue realmente trabajado por el equipo. La presentación lo contrasta con los períodos en colas, buffers, bloqueos o estados en los que nadie está trabajando sobre el elemento.

Si una historia permanece varios días en «En desarrollo» pero pasa dos días bloqueada, esos días de bloqueo no deberían considerarse trabajo efectivo. Las columnas de un tablero ayudan a interpretar dónde se consume tiempo, aunque registrar trabajo efectivo con precisión puede requerir datos adicionales.

4.4. Eficiencia del ciclo de proceso

La presentación formula la eficiencia de ciclo como Touch Time / Elapsed Time. Para expresarla como porcentaje debe multiplicarse por 100:

Eficiencia del ciclo (%) = (Touch Time / Elapsed Time) × 100. Aquí, «Elapsed Time» representa el tiempo transcurrido total del ciclo considerado; se debe dejar explícito el intervalo que se está usando.

Ejemplo ilustrativo: una tarea tarda 10 días desde el inicio hasta su finalización (Cycle Time) pero recibe trabajo efectivo durante 4 días. Eficiencia = 4 / 10 × 100 = 40 %. El 60 % restante corresponde al tiempo en que el trabajo no estuvo siendo procesado activamente.

Una eficiencia baja puede sugerir esperas o bloqueos relevantes, pero necesita interpretarse junto al tipo de trabajo. No basta por sí sola para identificar la causa.

4.5. Ejemplo completo: las cuatro medidas

Un cliente solicita una funcionalidad el día 1. El equipo comienza a trabajarla el día 4, dedica 3 días de trabajo efectivo y la entrega el día 11. Si medimos el tiempo transcurrido utilizando las diferencias entre esos hitos:

- Lead Time: 11 − 1 = 10 días desde el pedido hasta la entrega.
- Cycle Time: 11 − 4 = 7 días desde que comenzó el trabajo hasta su entrega.
- Touch Time: 3 días de trabajo efectivo.
- Eficiencia: (3 / 7) × 100 ≈ 42,9 % del tiempo del ciclo fue trabajo efectivo.

> Para entenderlo: En el tablero ilustrado en la diapositiva 16, las columnas Backlog, Dev, Test, Deploy y Complete muestran visualmente por qué Lead Time abarca un tramo mayor del recorrido que Cycle Time.

5. Métricas orientadas al servicio en Kanban

La diapositiva 19 diferencia cuatro nociones asociadas con la percepción del servicio y la capacidad real de entrega. Conviene aprenderlas por su función, no solo por las siglas que suelen acompañarlas en otros materiales.

- Expectativa de nivel de servicio: plazo o calidad de respuesta que los clientes esperan recibir.
- Capacidad del nivel de servicio: nivel que el sistema demuestra ser capaz de entregar con su funcionamiento real.
- Acuerdo de nivel de servicio: compromiso de prestación acordado explícitamente con el cliente.
- Umbral de adecuación del servicio: punto por debajo del cual el servicio resulta inaceptable para el cliente.

Ejemplo ilustrativo: un cliente espera recibir correcciones en cuatro días; los registros del flujo muestran que el equipo suele completarlas en seis. El acuerdo que pueda negociarse debería considerar tanto las expectativas como el desempeño observado, y existe un límite por debajo del cual la experiencia del cliente se vuelve inaceptable.

6. Comparación de métricas según el enfoque

La diapositiva 20 resume las medidas típicas de cada perspectiva. No se trata de que unas anulen a las otras, sino de que cada enfoque da prioridad a preguntas distintas.

Enfoque	Métricas de la presentación	Pregunta que ayudan a responder
Tradicional	Esfuerzo, tiempo, costos, defectos	¿Cómo avanzamos frente a lo estimado y qué calidad presenta el resultado?
Ágil	Velocidad, capacidad, Running Tested Features	¿Cuánto trabajo termina el equipo, de qué disponibilidad parte y qué funciones verificadas están operativas?
Lean / Kanban	Lead Time, Cycle Time, Touch Time, eficiencia del proceso	¿Cuánto tarda la entrega, cuánto dura el ciclo y cuánto tiempo se aprovecha en trabajo efectivo?

Por ejemplo, un equipo puede necesitar simultáneamente una métrica de costos para el presupuesto, una de RTF para valorar el avance del producto y un Lead Time para mejorar la experiencia de entrega. Elegir indicadores depende de las decisiones reales que debe tomar.

7. Métricas del producto software

Además de mirar el proceso y el proyecto, la presentación identifica dos grandes familias de métricas directamente relacionadas con el producto: tamaño y defectos.

7.1. Métricas de tamaño

- Líneas de código: cuantifican código fuente. No deben interpretarse automáticamente como valor aportado o productividad.
- Requerimientos: cantidad de necesidades o condiciones especificadas para el producto.
- Casos de uso: interacciones o escenarios funcionales descritos desde la perspectiva de los actores.
- Features: funcionalidades o capacidades del producto.
- Clases: unidades estructurales de código en sistemas orientados a objetos.
- Puntos de historia: unidades de estimación relativa de trabajo empleadas por un equipo ágil.

La diapositiva 21 agrupa casos de uso, features, clases y puntos de historia bajo la rama de requerimientos. En la práctica, estas son unidades distintas, por lo que conviene explicitar qué se está contabilizando antes de compararlas.

7.2. Métricas de defectos

- Cobertura: proporción del producto o de su comportamiento alcanzada por las pruebas, según el criterio definido. La cobertura de código no prueba por sí sola que las pruebas sean buenas.
- Defectos por severidad: distribución de fallas según su impacto o gravedad. Distinguir defectos críticos de menores facilita priorizar acciones.
- Densidad de defectos: cantidad de defectos relativa a alguna medida de tamaño. Fórmula general: defectos / unidad de tamaño definida.

Ejemplo ilustrativo: si se encuentran 12 defectos en 6.000 líneas de código, la densidad equivale a 2 defectos por cada 1.000 líneas. Para interpretar el indicador deben mantenerse consistentes la unidad, el

período y el criterio de conteo de defectos.

8. Idea final: comprender antes que juzgar

La presentación cierra con la invitación «Compara herramientas para entender, no para juzgar» y pregunta cuál es mejor. La imagen de distintos utensilios refuerza que una herramienta es útil dependiendo del propósito; lo mismo ocurre con las métricas.

Un único valor no resume toda la salud de un proyecto. Para interpretar una métrica es necesario conocer su definición, período de medición, contexto y las decisiones que puede respaldar. La mejor medida es la que hace visible una situación relevante y permite actuar sobre ella, no necesariamente la que genera el número más grande o el tablero más complejo.